

Low-Compilation-Cost Register Allocation in LLVM-Based Binary Translation

Xiangwei Meng
Lanzhou University
Lanzhou, China

Wei Li
Tsinghua University
Beijing, China

Chen Gao
Lanzhou University
Lanzhou, China

Fengyuan Ren*
Lanzhou University
Lanzhou, China
Tsinghua University
Beijing, China

Abstract

Efficiently allocating guest architecture registers to host registers is a crucial technique for improving performance in dynamic binary translation. Translators typically enlarge code regions to reduce context-switching overhead and expose more optimization opportunities. Traditional approaches load guest registers upon entering a code region and save them upon exiting, aiming to retain guest registers in host registers for as long as possible. However, as the size of code region increases, the growing number of intermediate variables and the increasing complexity of control flow lead to higher computational costs for register allocation, thereby increasing compilation overhead.

This paper investigates the causes of increased register allocation overhead in LLVM-based binary translators and proposes a low-compilation-cost strategy called LCCRA. It explicitly restores the semantics of accessing physical guest registers in LLVM IR, facilitating efficient register allocation on mainstream processor architectures. LCCRA propagates guest register values using virtual registers within each basic block. For values that need to cross basic block boundaries, virtual registers are introduced as substitutes for load operations where necessary, and redundant store operations are eliminated based on control flow analysis. Since this optimization is performed at the LLVM IR level, it does not interfere with existing LLVM optimizations, making it applicable to various LLVM-based binary translators.

We implement our strategy in CrossDBT, an LLVM-based binary translator. Evaluation on PARSEC benchmark suite,

considering varying code region sizes across two translation scenarios ($x86-64 \rightarrow x86-64$ and $x86-64 \rightarrow AArch64$), demonstrates that LCCRA achieves a 5.76%-7.79% reduction in end-to-end latency and a 69.55%-74.98% reduction in register allocation time when host registers are limited. In register-abundant scenarios, LCCRA still delivers slightly better end-to-end latency while consistently reducing compilation overhead.

CCS Concepts: • Software and its engineering → Retargetable compilers; • Computer systems organization → Architectures.

Keywords: Binary Translation, LLVM, Register Allocation

ACM Reference Format:

Xiangwei Meng, Chen Gao, Wei Li, and Fengyuan Ren. 2026. Low-Compilation-Cost Register Allocation in LLVM-Based Binary Translation. In *21st European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3767295.3803591>

1 Introduction

Binary translation enables migrating binary programs from one instruction set architecture (ISA), referred to as the guest ISA, to another, the host ISA [31], and has widely employed in cross-ISA software migration, virtualization, and software analysis. With the growing prevalence of ARM processors in the market, there is an increasing demand to migrate programs from mainstream x86-64 to AArch64 architecture to maintain a sound software ecosystem for ARM processors.

Many binary translators use intermediate representation (IR) as a bridge for instruction conversion to support multiple guest and host ISAs, allowing developers to focus on optimizing IR regardless of architecture-specific features. The quality of IR directly affects the performance of binary translation. LLVM [26, 32] is a comprehensive and robust compiler framework that provides a powerful foundation for developing and optimizing IR-based translation tools. In recent years, LLVM-based binary translators, such as HQEMU [20] and LnQ [23], have gained widespread attention due to LLVM's superior performance in the realm of compilers.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland Uk

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3803591>

Mainstream binary translators construct larger code regions to reduce context switching and expose more optimization opportunities. For example, DynamoRIO [7] employs trace to cache frequently executed paths. HQEMU [20] combines trace and leverages LLVM for extensive optimizations. Zhao et al. [43] lengthening traces in StarDBT [41] to improve opportunities for optimization. In this context, the efficient handling of guest registers becomes particularly critical, as they are central to preserving program semantics across translation and are frequently accessed throughout execution. In LLVM-based binary translation, guest registers are commonly modeled as memory locations, either through fixed memory addresses or stack slots allocated via `alloca`. This memory-based modeling leads to frequent load and store instructions in the LLVM IR. To improve runtime performance, these memory operations are often promoted to SSA variables to facilitate optimization and register reuse. When this promotion is applied over large code regions, it introduces a large number of long-lived virtual registers and increases interference among them, significantly raising the complexity of register allocation at compile time. As a result, compilation time grows rapidly with region size, limiting the ability to meet low-latency responsiveness requirements in dynamic binary translation.

This paper empirically observes the compilation overhead caused by register allocation in large code regions and proposes LCCRA, a lightweight strategy that avoids full-scale register promotion. By selectively propagating values, LCCRA reduces compilation cost while retaining runtime benefits, better meeting the real-time requirements of dynamic binary translation. We apply LCCRA in CrossDBT [27], a LLVM-based binary translator, and then validate it through experiments on PARSEC benchmark [6].

The main contributions are as follows:

- Identifying the compilation overhead challenge in LLVM-based binary translation when operating on large code regions, and analyzing its relation to register promotion strategies.
- Proposing a low-compilation-cost strategy that reduces guest-register memory accesses in LLVM IR, guides the LLVM register allocator to operate at lower complexity and overhead, and reduces the computational cost of register allocation.
- Implementing the strategy in an LLVM-based binary translator CrossDBT, we demonstrate through our experiments that LCCRA provides benefits in register-constrained scenarios (e.g., x86-64 $\rightarrow$ x86-64). It also maintains slightly better performance with reduced compilation costs even when host registers are abundant (e.g., x86-64 $\rightarrow$ AArch64), confirming its robustness and practicality for dynamic translation.

2 Background and Motivation

2.1 Binary Translation

Binary translation converts binary code from one ISA to another, aiming to conduct software migration without source code [10, 39]. Depending on whether the translation is performed at runtime, binary translation can be classified into static binary translation (SBT) and dynamic binary translation (DBT). SBT cannot handle runtime situations such as indirect branches and self-modifying code precisely, thus DBT is a preferred choice for practical software migration [10, 36]. SBT is more commonly used for static analysis of binary code [5, 18, 19].

IR-based binary translators lift guest binary instructions into IR, optimize them, and then compile them into target instructions [4, 9, 11, 20, 22, 24, 27, 36]. Compared to non-IR-based binary translator [3, 12, 25, 31, 40] that often rely on direct instruction mapping, IR-based translation enables higher-quality target code and simplifies handling ISA differences. For example, QEMU [4] uses TCG IR, HQEMU [20] further extends QEMU by converting TCG IR into LLVM IR which leverages LLVM's in-built optimizations to produce enhanced IR. IR-based binary translators allow developers to focus on optimizing IR, significantly reducing the development complexity caused by differences between guest and host ISAs, while facilitating the quality of target instructions. By combining code caching and other techniques, these translators effectively mitigate translation overhead.

CrossDBT [27] is a LLVM-based dynamic binary translator that directly translates guest instructions into LLVM IR, which is then compiled into target architecture instructions and executed using LLVM Just-In-Time (JIT) compiler [35]. Most LLVM-based binary tools typically use multiple IRs (e.g. HQEMU [20] also uses TCG IR) and focus on translation within the same architecture (e.g. BinRec [1] and Instrew [15]). Since they involve mixed IR and unlikely provide practical environments to conduct investigation in this work, CrossDBT is selected as the experimental platform.

2.2 LLVM

LLVM [26, 32] is a modular and reusable compiler framework, whose core component is LLVM IR designed in the static single assignment (SSA). The design of LLVM IR shares some characteristics with RISC. The flexible design allows for a wide range of optimizations and being well-suited for diverse target ISAs. A LLVM pass [34] is a process that operates on LLVM IR, and LLVM provides numerous passes for transforming and analyzing LLVM IR, which facilitates the generation of high-quality target instructions. Additionally, LLVM supports a wide range of back-end architectures, and includes LLVM JIT for the real-time compilation of LLVM IR, thus particularly well-suited for DBT scenarios where dynamic code generation is required.

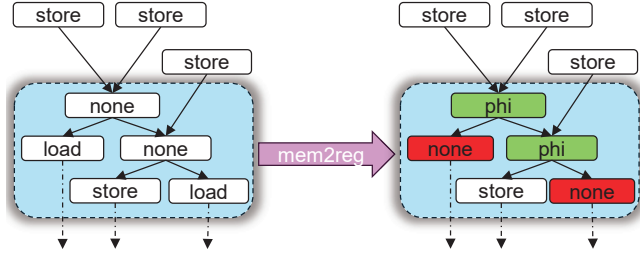


Figure 1. A control flow illustration for applying mem2reg pass.

LLVM provides the mem2reg pass to promote stack-allocated variables to registers by utilizing phi nodes [33, 34]. A phi node is used to select an incoming value based on the predecessor blocks from which control flow originates, effectively merging values from different paths in a control flow graph at the point of convergence. The mem2reg pass promotes `alloca` instructions used solely for load and store operations to register operations, and uses phi nodes to correctly select and propagate values across basic blocks in the LLVM IR in accordance with the control flow, thereby avoiding memory accesses.

2.3 Translation Granularity and Register Allocation Challenges

By grouping larger code segments into unified optimization units, modern binary translators gain access to optimization opportunities that go beyond the scope of individual basic blocks. Initially, **basic block** was widely adopted due to their simplicity and well-defined control flow boundaries, as seen in early dynamic binary instrumentation frameworks such as early Valgrind [28]. To improve optimization potential and reduce context switching, many systems later adopted **trace**, which is linear sequence of frequently executed basic blocks, as the primary translation unit, as exemplified by DynamoRIO [7] and HQEMU [20]. Beyond traces, various other translation units have been explored. **Superblock**, which extend traces by allowing multiple entries but only one exit, was used in Valgrind [28] and CrossBit [17], facilitating predication and scheduling optimizations. Other units, such as **Megablock**, have been employed in systems like the one described in [30].

As the translation unit expands, it tends to cover more guest instructions, resulting in more frequent accesses to guest registers. This increased register activity places greater demands on register allocation, which becomes crucial for reducing memory accesses and realizing the full benefits of LLVM optimization. However, traditional optimization strategies for register allocation, especially those targeting physical registers, are not readily applicable in LLVM-based binary translators. This is primarily due to the SSA form and the high abstraction level of LLVM IR, which decouple the

IR from low-level register semantics and hinder the direct application of such optimizations.

A widely used approach in LLVM-based translators is combining `alloca` with mem2reg pass [8, 36, 38, 42]. In this approach, guest registers are first represented by stack slots via `alloca` instructions, and then promoted to SSA variables by mem2reg pass. As a result, these variables can be efficiently mapped to host registers through LLVM’s standard register allocation, avoiding unnecessary memory accesses. Although this method was not originally intended for binary translation, it often delivers near-optimal performance by enabling SSA conversion and facilitating LLVM’s powerful optimizations. For this reason, we regard it as a practical performance reference.

2.4 Motivation

While improving register allocation quality might seem a natural direction to reduce memory accesses, fundamental limitations constrain its potential. First, binary translation, especially from CISC architectures, introduces numerous intermediate variables, due to the need to decompose a single guest instruction into multiple RISC-like IR instructions. These intermediate values compete with guest registers for limited host registers. Second, when the number of host registers is no greater than that of guest registers, it is fundamentally impossible to keep all guest values in registers, regardless of allocation strategy. Finally, larger regions bring more complex control flow, making it difficult to design practical allocation algorithms that outperform LLVM’s built-in strategies. These structural issues suggest that optimizing register allocation quality alone yields diminishing returns.

Although the `alloca` and mem2reg combination achieves desirable performance, the associated compilation time overhead should not be overlooked. As shown in Figure 1, mem2reg pass eliminates two load instructions (red blocks) by replacing them with two phi nodes (green blocks). This transformation reduces redundant memory accesses, potentially improving performance. However, this pass can introduce compilation overhead. For example, LLVM must strive to allocate a phi node (green blocks) and its incoming values, whether stored values or phi nodes from predecessor blocks, to the same host register, in order to avoid additional `mov` or `load/store` instructions in the final binary. Meanwhile, LLVM attempts to allocate registers for phi nodes in the green blocks, which implicitly extends the live range of guest registers and increases register pressure. Additionally, in the presence of multiple predecessors, the binary code required to implement the semantics of phi nodes can incur noticeable compilation cost. These facts pose challenges to dynamic translation with real-time constraints.

These observations motivate us to shift our focus. Rather than attempting to surpass the performance of combining `alloca` with mem2reg through complex heuristics or high-overhead allocation strategies, we aim to approximate its

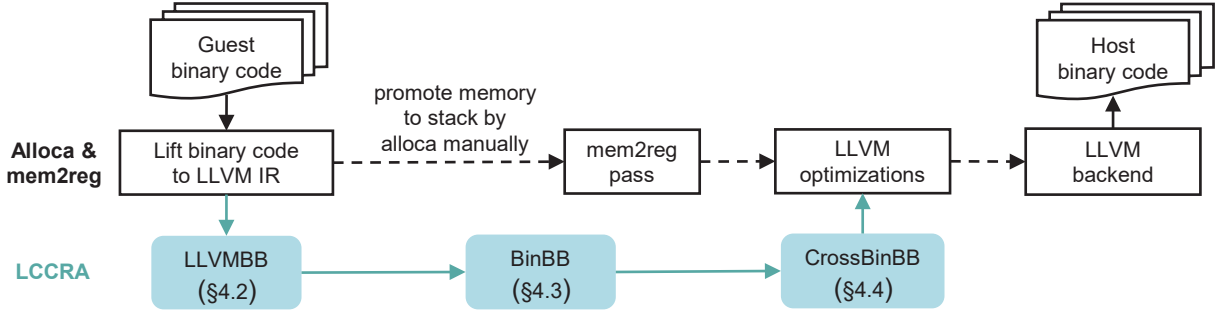


Figure 2. Overview of LCCRA.

performance through a lightweight approach, minimizing translation overhead, which tailored to scenarios like dynamic binary translation.

3 Basic Idea

A key advantage of LLVM-based binary translators lies in their portability across different host architectures. To preserve this flexibility, we avoid introducing any architecture-specific modifications into LLVM. Additionally, we refrain from employing non-SSA techniques to eliminate redundant memory accesses, so as to stay compatible with LLVM’s optimization pipeline. Since phi nodes are LLVM’s canonical mechanism for eliminating redundant loads in SSA form, our method follows the principle of mem2reg pass, but applies it in a more controlled and lightweight manner, taking into account the characteristics of binary translation.

Our method is based on the following four ideas:

(1) **Avoiding the use of alloca:** Instead of modeling guest registers as stack-allocated memory and relying on mem2reg, we directly manipulate load and store instructions to maintain the semantics of guest registers. This avoids introducing memory accesses spanning multiple basic blocks and enables fine-grained control over where and when memory operations occur.

(2) **Promotion within guest basic blocks:** We eliminate redundant memory accesses by applying mem2reg-like logic confined to individual guest blocks. This includes replacing some load instructions with phi nodes and pruning unnecessary store instructions, without introducing inter-block phi node propagation.

(3) **Handling cross-basic-block register values with limited propagation:** When promoting guest register values across LLVM basic blocks, we selectively introduce phi nodes. However, not all phi nodes yield equal benefit. In particular, those whose values are not used until several basic blocks later (a metric we term *first-use block distance*, i.e., the number of basic blocks between the definition of a value and its first use) can lead to inefficiencies. Keeping such values in host registers increases register pressure, while spilling

them back to memory undermines the purpose of promotion. Either approach complicates the generated code and increases compilation overhead. To address this, we adopt a heuristic strategy that restricts the creation of phi nodes with excessively long first-use block distances across LLVM basic blocks. By imposing a distance threshold, we aim to avoid introducing phi nodes that contribute limited benefit yet impose considerable compilation and register pressure.

(4) **Control-flow-sensitive store handling across guest basic blocks:** We explicitly remove store instructions that are guaranteed to be overwritten by subsequent stores along all control-flow paths, leveraging the precise control-flow knowledge available during translation.

Additionally, an important aspect of this process is to ensure register semantic consistency across different load/store operations. Specifically, when binary lift tools or other front-end techniques lift guest instructions to LLVM IR, accesses to the same guest register at different widths (e.g., load i64 vs. load i32) may be allocated to different registers in LLVM. To resolve this, we unify these operations at the IR level by ensuring that all load and store accesses are treated as the same full-width operation (e.g., load i64 for general-purpose register in x86-64), and then perform necessary operations to extract the relevant part of the value required for the corresponding IR. Without such normalization, LLVM may treat different-width operations as independent variables, fragmenting the register state. LLVM can allocate such variables to the same host register (when the host architecture allows sub-register aliasing), ensuring that phi nodes correctly merge accesses to the same guest register, preventing redundancy allocation and enabling effective optimizations.

4 Design of LCCRA

4.1 Overview

LCCRA processes a series of transformations on the LLVM IR produced by lifting tools to reduce load/store operations on guest registers introduced during lifting, without altering semantics, and to guide more efficient register allocation on the target machine. The transformed IR will be optimized and

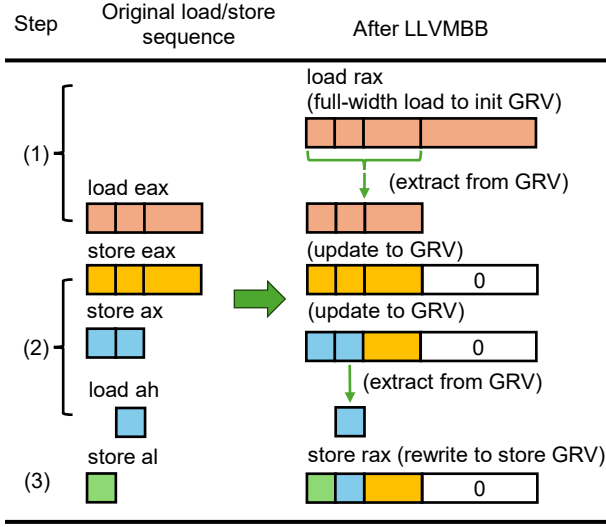


Figure 3. A transformation demo of LLVMBB focused on load/store of guest register RAX.

compiled by LLVM later. Figure 2 illustrates the workflow of LCCRA. We first preprocess LLVM IR, aiming to parse the load/store instructions associated with the guest registers in LLVM IR. Additionally, in cases with load merging or store merging, we decompose merged load or store instructions into multiple scalar load or store operations, similar to the scalar replacement of aggregates (SROA) pass [34] in LLVM.

Next, the LLVM IR undergoes further processing through three stages. The first stage (referred to as *LLVMBB*) unifies the access width of guest registers to expose aliasing relations among memory accesses corresponding to guest registers. The second stage (referred to as *BinBB*) performs localized load/store elimination within each guest basic block by constructing phi nodes and removing overwritten stores. The final stage (referred to as *CrossBinBB*) replaces inter-block load operations with appropriate phi nodes, enabling value propagation across LLVM basic blocks, and then prunes redundant store operations based on inter-block control flow.

The workflow is a sequential process, where each stage relies on the successful execution of the previous stages. For instance, when discussing *BinBB*, it is understood that *LLVMBB* has already been completed.

4.2 LLVMBB

This stage unifies the width of all memory operations that emulate guest registers in the lifted IR. When using binary lift tools or similar frontends, accesses to the same guest register may be translated into loads and stores of different widths (e.g., i32, i64), which prevents LLVM from recognizing them as related. We address this by replacing all such accesses with full-width loads and stores, and computing the required partial values using general data manipulation operations.

Algorithm 1: LLVMBB Transformation

Input: LLVM basic block BB, guest register R, maximum width W of R (in bits)
Output: Modified BB with unified full-width accesses for R

```

1  S ← CollectLoadStoreSequence(BB, R);
2  if S is empty then
3    return;
4  T ← LLVMTypeForWidth(W);
5  hasStores ← any(IsStore(instr) for instr in S);
6  if hasStores then
7    insertPosAfterLastStore ←
      After>LastStoreInstructionIn(S);
  /* Step (1): Initialize GRV */
8  if S not empty and IsStore(S[0]) and
   GetWidth(S[0]) = W then
9    currentGRV ← GetStoredValue(S[0]);
10   RemoveInstruction(S[0]);
11 else
12   loadInstr ← CreateLoad(T, AddressOf(R));
13   InsertBefore(loadInstr, FirstInstructionIn(S));
14   currentGRV ← loadInstr;
  /* Step (2): Transform sequence */
15 foreach instr in S do
16   bitStart ← GetBitStartForAccess(instr);
17   accessWidth ← GetWidth(instr);
18   if IsLoad(instr) then
19     extracted ← ExtractFromGRV(currentGRV,
20                               bitStart, accessWidth);
21     ReplaceUses(instr, extracted);
22     RemoveInstruction(instr);
23   else
24     val ← GetStoredValue(instr);
25     currentGRV ← UpdateGRV(currentGRV,
26                             val, bitStart, accessWidth);
27     RemoveInstruction(instr);
  /* Step (3): Add final store if needed */
28 if hasStores then
29   storeInstr ←
30     CreateStore(currentGRV, AddressOf(R));
31   InsertAfter(storeInstr, insertPosAfterLastStore);

```

This full-width value, referred to as the guest register value (refer to as GRV), represents the value of the guest register across all accesses.

Figure 3 and Algorithm 1 illustrates the transformation of LLVMBB. The transformation process for the preprocessed load/store sequence in each LLVM basic block is as follows:

1	<i>; Original IR</i>	<i>; IR after BinBB</i>
2	BB0:	BB0:
3	load rax	load rax
4	;	load rcx
5	;	load rdx
6	br <rflag> BB1, BB2	br <rflag> BB1, BB2
7	BB1:	BB1:
8	load rax	;
9	load rcx	;
10	load rdx	;
11	store rcx	;
12	store rax	store rax
13	br BB3	br BB3
14	BB2:	BB2:
15	load rdx	;
16	load rcx	;
17	store rcx	;
18	store rdx	store rdx
19	br BB3	br BB3
20	BB3:	BB3:
21	load rcx	rcx = phi
22	;	store rcx
23	exit	exit

Figure 4. A LLVM IR demo of BinBB focused on load/store.

(1) First, a load instruction for the guest register at its maximum width is added at the beginning of the sequence to initialize the GRV (line 8 - line 10 in Algorithm 1). If the first instruction is a store at the maximum width, this step is unnecessary since the stored value will directly become GRV (line 11 - line 14).

(2) The load/store sequence is then traversed sequentially. When a load instruction is encountered, it is replaced with GRV or a variable derived from some operations on GRV (line 18 - line 22). When a store instruction is encountered, GRV is updated according to the position and width of the store operation, and the store instruction is removed (line 23 - line 27).

(3) If the initial load/store sequence contains store operations (line 5 - line 7), the last store operation is rewritten based on GRV to store at the maximum width (line 28 - line 30).

In this stage, GRV functions as the guest register, with operations like `trunc` and `insertElement` simulating the partial load and store behaviors of guest instructions. Although we do not assume any target-architecture-specific knowledge (such as register aliasing rules), LLVMBB can take advantage of the architectural similarities commonly found in modern ISAs, such as 64-bit general-purpose registers and partial access to overlapping subregisters. These similarities allow LLVM's register allocator to naturally and efficiently map guest registers to host registers. In the absence of such similarities, splitting guest register values across multiple host registers becomes unavoidable, regardless of whether

the translator explicitly tracks physical registers or not. Additionally, LLVMBB reduces the number of accesses to each guest register to at most one full-width load and one full-width store per basic block. The handling of endianness is determined at the lifting stage and is orthogonal to LLVMBB.

4.3 BinBB

For the LLVM basic block(s) translated from a single guest basic block, we note that although a guest block is often mapped to a single LLVM basic block, certain guest instructions (such as conditional moves (e.g., `cmov`) or other operations) that introduce implicit control flow, unavoidably cause the translation to be split into multiple LLVM basic blocks. In either case, we hoist load instructions to the entry block and delay store operations to as late as possible.

Throughout this process, the GRV is explicitly maintained and updated to reflect guest register semantics. Load instructions in non-entry basic blocks are replaced by either directly reusing GRV passed from their unique predecessor, or introducing phi nodes when multiple predecessors exist. Store instructions are treated as updates to the GRV and are delayed accordingly. Because the control flow within such translated LLVM basic blocks is generally simple and involves only a small number of variables, the number of phi nodes and the number of incoming values per phi node tends to remain small. This localized use of phi nodes reduces the complexity of register assignment and helps contain register allocation pressure within individual basic blocks, rather than escalating it across larger translated code region. The phi node placement in this stage follows the SSA promotion strategy adopted by `mem2reg` pass.

Figure 4 shows an example of transformation processed by BinBB. We focus on load/store sequence for some guest registers. Since the widths of load/store are same now, we omit type of operand of load/store instructions. Original loads (Line 8, 9, 10, 15 and 16) are advanced into BB0 (Line 4 and 5), while the IR requiring load in BB1 and BB2 can use the value from BB0 now. Original stores (Line 11 and 17) are delayed to BB3 (Line 22). The original load in BB3 (Line 21) can be replaced by phi node (Line 21) which selects value from original stores (Line 11 and 17).

While this transformation temporarily extends the live ranges of loads and stores, later LLVM optimizations will effectively reschedule them, ensuring efficient register usage and eliminating unnecessary holding of values.

4.4 CrossBinBB

In this stage, we handle memory operations that span across multiple LLVM basic blocks, addressing both load and store instructions for guest register values. We first focus on load promotion and subsequently perform store elimination, as the former facilitates more effective pruning of the latter.

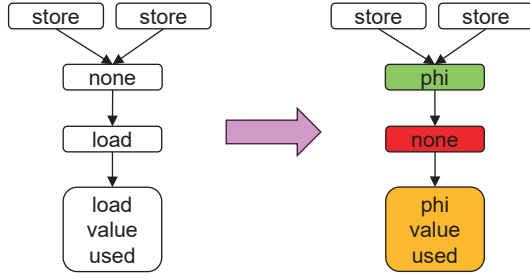


Figure 5. Example of phi node generation to replace a load instruction (first-use block distance = 2).

To reduce redundant loads, we attempt to lift them into SSA form via phi nodes at control flow joins. However, indiscriminate phi node insertion may increase register pressure or compilation time, especially when the first use of a phi node value is far from its definition. To mitigate this, we apply a usage-distance-guided strategy that selectively inserts phi nodes only when their first use lies within a limited number of basic blocks. For definitions with long-distance uses, we fall back to memory operations to avoid excessive register pressure.

To further control analysis complexity, we impose an additional constraint during phi node insertion: each predecessor block must contain at least an access to guest register value. This prevents the need for recursive forward searches when identifying value definitions across control-flow paths. Moreover, this constraint ensures the necessity of the inserted phi node, as it implies that the value is frequently accessed across multiple control-flow paths and thus merits promotion.

We restrict the insertion of phi nodes to cases where the first use of the corresponding guest register occurs within a short control-flow block distance from its definition. For illustration, Figure 5 shows an example with a distance threshold of two successor blocks. This prevents unnecessary propagation of register values across distant or rarely used paths and avoids analysis complexity arising from recursive predecessor analysis.

After reducing load-related memory accesses, we analyze the remaining store instructions, which serve to preserve guest register state across basic blocks. If a GRV is consistently overwritten along all control paths before any subsequent load instruction, the earlier store becomes redundant and can be eliminated. Stores in basic blocks that may lead to exits from translated code are conservatively preserved to ensure semantic correctness.

4.5 Takeaways

Our design adopts a staged and selective strategy for promoting memory accesses to guest register to virtual registers. Instead of applying `mem2reg` uniformly across large regions,

we limit the insertion of phi nodes based on first-use distances and control-flow structure. This approach avoids two significant issues: first, the generation of long-distance phi nodes that span across many basic blocks; and second, the high complexity of register allocation when phi nodes are inserted into blocks with many predecessors. While this complexity remains inherent to SSA form, our heuristics effectively reduce the number of such phi nodes, thereby alleviating the allocator pressure and improving compilation efficiency. Combined with subsequent store elimination guided by control-flow-aware analysis, this design not only reduces redundant memory accesses but also mitigates register allocation pressure and compilation overhead.

While our strategy enables effective elimination of redundant loads via targeted phi node insertion, the optimization of stores remains more limited. This is primarily because we do not aggressively promote all loads to phi nodes as in `mem2reg`, thereby retaining possible load-after-store dependencies that prevent store removal.

LCCRA is designed to be semantics-preserving and to strictly follow SSA form and LLVM type constraints. Consequently, the transformed IR remains well-formed and fully compatible with downstream LLVM optimization.

5 Evaluation

5.1 Experimental Setup

The experiments are conducted on **CrossDBT** [27] configured with LLVM version 15.0.7. The LLVM optimizer is configured with the `-O3` level. Additional hardware configurations and details of the benchmark are listed below.

Testbed involves translating and executing benchmark programs, which are compiled for x86-64 platform, on itself and another AArch64 machine. LCCRA is applied to most used registers of x86-64: general-purpose registers (GPRs), flag registers (RFLAGS), vector registers and the instruction pointer register (RIP).

- **x86-64:** A server equipped with two Intel Xeon Silver 4310 processors (12 cores per chip, 2.1 GHz) and 192GB memory, running Ubuntu 20.04 LTS with Linux kernel 6.8.
- **AArch64:** A server equipped with two Huawei Kunpeng 920-4826 processors (ARMv8.2-a, 48 cores per chip, 2.6GHz) and 192GB memory, running Ubuntu 20.04 LTS with Linux kernel 5.4.

Benchmark. We use the PARSEC 3.0 benchmark suite [6] in our evaluation. Two benchmarks, `raytrace` and `x264`, are excluded as they do not execute correctly on our evaluation platform, even natively, and thus cannot be used for comparison in binary translation experiments.

Translation Granularity. To evaluate our register allocation method under more realistic translation conditions, we adopt a trace-based region formation strategy as the translation unit. To study the effectiveness of our method on large

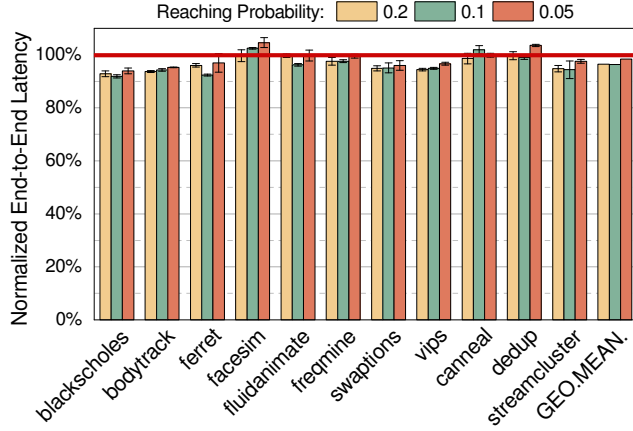


Figure 6. End-to-end latency of PARSEC benchmarks for x86-64 $\rightarrow$ AArch64 translation. All results are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate better performance.

code regions, we relax trace formation parameters in our experiments, allowing traces to cover more code. Similar to Hong et al. [21], we construct execution traces to guide region formation, with traces obtained from translation-time branch recording. Specifically, a node is considered hot if its execution frequency exceeds 32, and a trace is terminated if it grows beyond 256 basic blocks. In addition, we control trace expansion by varying the reaching probability threshold, which determines whether a control-flow successor is included in the trace. We evaluate three thresholds (0.2, 0.1, and 0.05) to generate traces of varying sizes. Smaller reaching probability thresholds correspond to larger traces, as more successors are included. This setup enables us to systematically evaluate the impact of trace size on our register allocation method.

We evaluate our approach under two translation scenarios: x86-64 to AArch64 and x86-64 to x86-64. The former reflects a realistic and increasingly common use case in cross-architecture binary translation. The latter, while not typical in real-world deployment, serves as a stress test for our method under limited host register availability, enabling a clearer observation of its impact. The detailed evaluation results for each scenario are presented in §5.3 and §5.4, respectively.

To ensure a fair comparison, we apply stage LLVMBB as a pre-processing step to `alloca` + `mem2reg` schemes, normalizing register accesses by promoting all subregister-level loads and stores to full-width register accesses. This ensures that `mem2reg` operates on unified width register variables, consistent with its design expectations, and prevents it from being penalized by irregular or partial register usage patterns.

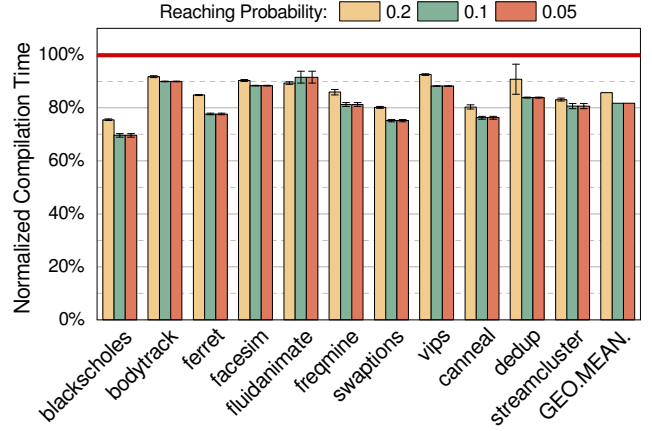


Figure 7. Compilation time of PARSEC benchmarks for x86-64 $\rightarrow$ AArch64 translation. All values are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate reduced compilation overhead.

5.2 First-use Block Distance

On the PARSEC benchmark suite, we evaluated the first-use distance parameter over 1, 2, 4, 8, 16. The results show that on x86-64 the maximum variation in end-to-end latency is about 1%–2%, while on AArch64 (with a larger register file) the effect is even smaller and often negligible.

Increasing the distance threshold enable more phi node insertions and store eliminations, which in principle can reduce runtime memory accesses. However, they also increase overhead, both because LCCRA itself must operate over a larger search space and because additional phi nodes extend variable live ranges, making register allocation more costly. These opposing effects largely cancel out, resulting in only marginal differences in end-to-end latency.

Two further factors contribute to the small runtime impact. First, the incremental number of loads replaced or stores eliminated decreases quickly as the threshold grows, limiting the potential performance benefit. Second, even when some stores remain because of later loads in certain successors, their impact on runtime performance is generally limited. In practice, modern CPUs can pipeline and buffer store operations effectively, so their contribution to execution path stalls or memory bottlenecks is limited. Taken together, these factors explain why larger thresholds provide little observable improvement.

Given this trend, we adopt distance = 1 as the default setting in subsequent experiments: it achieves end-to-end latency statistically indistinguishable from larger thresholds, while its conservative nature makes it more robust for other or previously unseen programs.

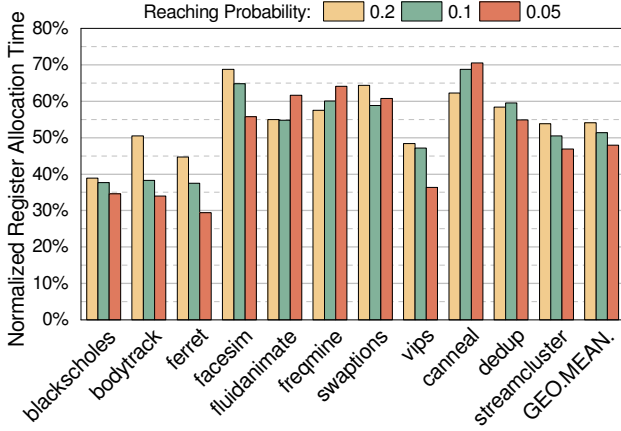


Figure 8. Time spent in register allocation during compilation of PARSEC benchmarks for x86-64 → AArch64 translation. All results are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate faster register allocation.

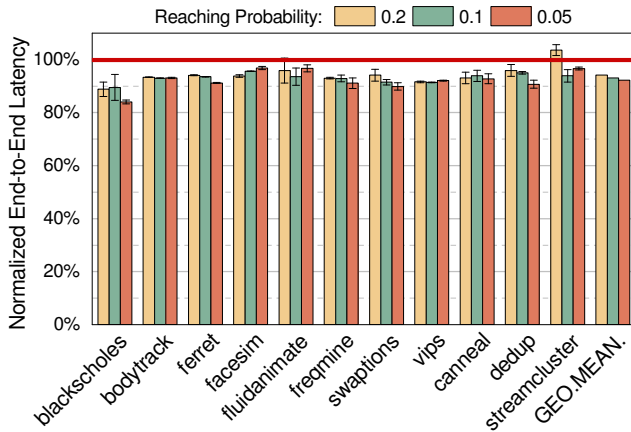


Figure 9. End-to-end latency of PARSEC benchmarks for x86-64 → x86-64 translation. All results are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate better performance.

5.3 x86-64 to AArch64

Figure 6 presents the normalized end-to-end latency of our approach (LCCRA) relative to the mem2reg baseline for x86-64 to AArch64 translation. Across most benchmarks, LCCRA achieves comparable or slightly lower latency than mem2reg. While mem2reg occasionally yields marginally better runtime performance due to more aggressive register promotion, this benefit is offset by its substantially higher compilation cost. This indicates that even when the host architecture provides more registers than the guest, LCCRA can still match mem2reg in overall latency. Moreover, as trace

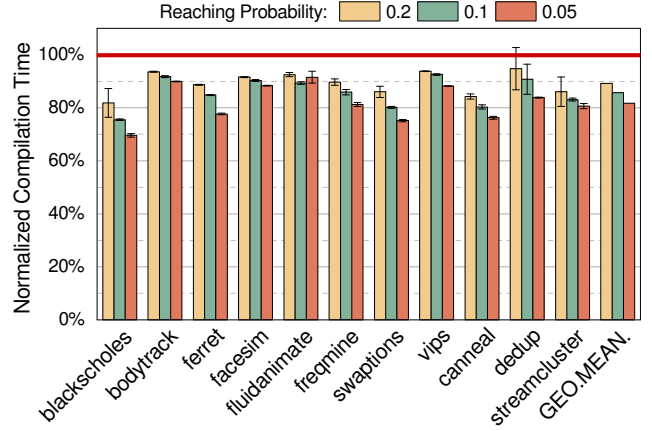


Figure 10. Compilation time of PARSEC benchmarks for x86-64 → x86-64 translation. All values are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate reduced compilation overhead.

sizes increase, LCCRA’s latency gradually approaches that of mem2reg, suggesting that LCCRA’s relative disadvantage in register utilization becomes more pronounced under larger code regions.

Figure 7 shows normalized compilation time. Since LCCRA avoids full-scale mem2reg promotion, compilation overhead is reduced by 14.29%–18.22%, making compilation efficiency one of LCCRA’s key strengths in dynamic translation scenarios. To further isolate the impact on the backend, we use llc (an LLVM static compiler tool) to measure the time spent specifically in the register allocation phase (greedy allocator). Figure 8 demonstrates that register allocation costs are reduced by more than 45.89%, further confirming that our method introduces lower complexity compared to conventional promotion.

Overall, these results demonstrate that LCCRA fulfills the central motivation of this work. It offers a lightweight yet effective register optimization technique that achieves meaningful improvements without sacrificing translation speed. LCCRA thus provides a practical alternative to full SSA-based pipelines, particularly in dynamic translation scenarios.

5.4 x86-64 to x86-64

Figure 9 presents the normalized end-to-end latency of LCCRA relative to the mem2reg baseline for x86-64 to x86-64 translation. In this setting, where host registers are more limited, LCCRA exhibits a clear advantage: end-to-end latency is reduced by 5.76%–7.79% compared to mem2reg. Importantly, this advantage grows as traces become larger, highlighting LCCRA’s ability to better manage register pressure when resources are constrained.

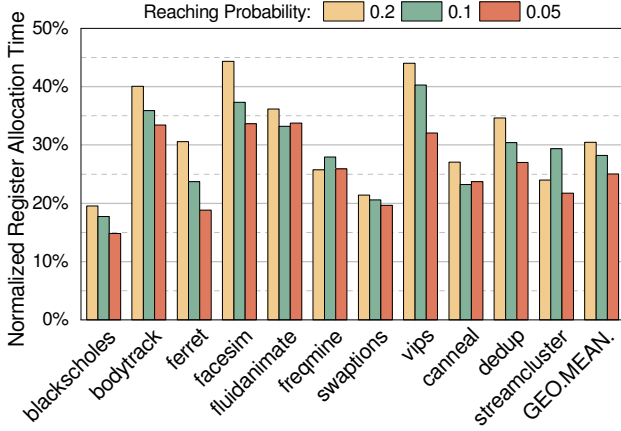


Figure 11. Time spent in register allocation during compilation of PARSEC benchmarks for x86-64 $\rightarrow$ x86-64 translation. All results are normalized to mem2reg, which is set to 100%. Smaller reaching probability corresponds to larger traces. Lower values indicate faster register allocation.

Figure 10 reports normalized compilation time. LCCRA reduces compilation overhead by 10.72%–18.22% relative to mem2reg. The reduction becomes increasingly pronounced with larger traces, reflecting the scalability of our approach in more demanding translation scenarios. As shown in Figure 11, register allocation costs are reduced by 69.55%–74.98%, a substantial improvement that directly stems from the reduced number and scope of promoted values. Unlike the AArch64 setting, where a larger register file partially offsets mem2reg’s cost, the tighter register constraints on x86-64 amplify LCCRA’s benefits.

Taken together, these results demonstrate that LCCRA not only reduces compilation overhead but also achieves tangible end-to-end gains when host registers are limited, with its advantages becoming more significant as trace size increases.

5.5 Takeaways

Despite compilation overhead generally decreasing relative to mem2reg as trace sizes increase, we observe differing trends in end-to-end latency across translation scenarios. In the x86-64 $\rightarrow$ AArch64 scenario, as trace size grows, LCCRA’s end-to-end latency gradually approaches that of mem2reg. This trend suggests that under larger code regions, mem2reg’s ability to exploit abundant host registers increasingly mitigates the disadvantages associated with full-scale register promotion, while LCCRA’s more conservative promotion strategy maintains latency comparable to mem2reg, with its relative advantage diminishing as trace size increases.

In contrast, for x86-64 $\rightarrow$ x86-64 translation with limited host registers, mem2reg incurs increasing compilation overhead as trace size grows, because LLVM attempts to

preserve guest register values in host registers. However, the shortage of registers prevents these values from being fully retained, resulting in frequent spills and reloads that do not translate into runtime performance gains. Under these conditions, LCCRA’s lightweight promotion strategy alleviates register pressure, reducing compilation cost while improving end-to-end latency. As traces expand, these benefits become increasingly pronounced, highlighting LCCRA’s advantage in register-constrained scenarios.

Overall, these observations highlight that the benefits of LCCRA are most pronounced when trace expansion genuinely increases register pressure, such as in host architectures with fewer available registers. LCCRA consistently provides comparable or improved latency while incurring lower compilation overhead, particularly in the register allocation phase. By reducing redundant guest register accesses through a lightweight approach, LCCRA achieves a favorable balance between optimization effectiveness and transformation cost. Its consistent behavior across different backends confirms portability and generality, establishing LCCRA as a practical and adaptable register allocation strategy for LLVM-based binary translation.

It is worth noting that in scenarios where programs execute for extended periods or repeatedly invoke the same code regions, the runtime performance benefits of mem2reg, stemming from more aggressive register promotion, may outweigh its higher compilation overhead. In such cases, mem2reg could achieve lower end-to-end latency despite its increased compilation cost, whereas LCCRA’s lightweight strategy primarily targets reduced compilation time and near-optimal register utilization, providing advantage in short-lived or dynamically translated workloads.

6 Related Work

6.1 LLVM-based Binary Tools

There are several binary translators based on LLVM [14, 20, 23, 36–38]. Most of them focus on implement a LLVM-based binary translator and achieve high performance by leveraging optimizations in LLVM. LnQ [23] utilizes a framework that integrates LLVM [26, 32] and QEMU [4], where QEMU serves as the emulation module and LLVM as the translation module. During translation, LnQ maintains a mapping between guest registers and LLVM registers using a register mapping table. HQEMU [20] integrates QEMU and LLVM, where QEMU handles the initial binary translation and LLVM optimizes the code in hot paths to produce high-quality translated code. HQEMU uses existing LLVM optimizations to eliminate redundant load/store instructions. LLBT [36] is a static binary translator based on LLVM. It declares guest registers as local variables via alloca and leverages the mem2reg pass to promote memory accesses to register operations.

DBrew-LLVM [16] rewrites binary code by leveraging standard LLVM optimizations, during which phi nodes are introduced unconditionally as part of SSA construction. Instrew [15] is a binary instrumentation tool that generates guest register facets on demand by deriving them from the whole facet, following a strategy conceptually similar to the mem2reg scheme shown in §5.

6.2 Lifting Binary to LLVM IR

Lifting binary to LLVM IR is a significant part in LLVM-based translators when facing issue in this work. LnQ [23] employs MCDisassembler class and other tools provided by LLVM to disassemble instructions in combination with pre-built LLVM IR templates to lift binaries. Lifting tools are also involved in some other binary tools, such as binary instrumentation and binary rewriting. Instrew [15] develops Rellume as the lifting tools for its binary instrumentation. There are other lifting tools, such as Mcsema [13, 29] and Retdec [2]. Most of them are designed mainly for static analysis and modification to binary files. They can be used in binary translators but have some performance and correctness issues when facing dynamic translation scenario.

Some other investigations generate LLVM IR by lifting from other IR. A typical work is HQEMU [20], which produces LLVM IR lifted from TCG IR generated previously by QEMU [4]. However, it is inevitable that redundant IR are generated by employing multi-layer IR in lifting tools, which constrains the upper limit of optimization and performance.

7 Conclusion

This paper presents a lightweight register promotion technique tailored for LLVM-based binary translation, which selectively promotes guest register accesses to virtual registers during translation. In contrast to traditional SSA promotion strategies such as mem2reg, our method avoids global code transformations and instead focuses on efficiently eliminating redundant memory operations with minimal translation overhead.

The proposed strategy demonstrates that it alleviates register allocation pressure, lowers end-to-end latency, and decreases compilation overhead, effectively addressing the tradeoff between code quality and translation latency in binary translators. This validates the practicality of lightweight, semantics-aware register handling in the context of fast binary translation.

Acknowledgments

We thank our shepherd, Yaniv David, and the anonymous reviewers for their valuable feedback. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant Nos. 62132007 and 62221003.

References

- [1] Anil Altinay, Joseph Nash, Taddeus Kroes, Prabhu Rajasekaran, Dixin Zhou, Adrian Dabrowski, David Gens, Yeoul Na, Stijn Volckaert, Cristiano Giuffrida, Herbert Bos, and Michael Franz. 2020. BinRec: dynamic binary lifting and recompilation. In *Proceedings of the Fifteenth European Conference on Computer Systems (Heraklion, Greece) (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 36, 16 pages. <https://doi.org/10.1145/3342195.3387550>
- [2] Avast Software. 2025. RetDec: A Retargetable Machine-Code Decompiler. <https://retdec.com/>.
- [3] L. Baraz, T. Devor, O. Etzion, S. Goldenberg, A. Skaletsky, Yun Wang, and Y. Zemach. 2003. IA-32 execution layer: a two-phase dynamic translator designed to support IA-32 applications on Itanium®-based systems. In *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. 191–201. <https://doi.org/10.1109/MICRO.2003.1253195>
- [4] Fabrice Bellard. 2005. QEMU, a Fast and Portable Dynamic Translator. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. USENIX Association, Anaheim, CA. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>
- [5] J. Bergeron, M. Debbabi, M.M. Erhioi, and B. Ktari. 1999. Static analysis of binary code to isolate malicious behaviors. In *Proceedings. IEEE 8th International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises (WET ICE '99)*. 184–189. <https://doi.org/10.1109/ENABL.1999.805197>
- [6] Christian Bienia. 2011. *Benchmarking modern multiprocessors*. Ph.D. Dissertation. Princeton University. <https://www.cs.princeton.edu/techreports/2010/890.pdf>.
- [7] Derek Bruening and Saman Amarasinghe. 2004. Efficient, transparent, and comprehensive runtime code manipulation. (2004).
- [8] Changbin Chen, Shu Sugita, Yotaro Nada, Hidetsugu Irie, Shuichi Sakai, and Ryota Shioya. 2025. Biotite: A High-Performance Static Binary Translator using Source-Level Information. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (Las Vegas, NV, USA) (CC '25)*. Association for Computing Machinery, New York, NY, USA, 167–179. <https://doi.org/10.1145/3708493.3712693>
- [9] Anton Chernoff, Mark Herdeg, Ray Hookway, Chris Reeve, Norman Rubin, Tony Tye, S. Bharadwaj Yadavalli, and John Yates. 1998. FX!32: A Profile-Directed Binary Translator. *IEEE Micro* 18, 2 (March 1998), 56–64. <https://doi.org/10.1109/40.671403>
- [10] Cifuentes and Malhotra. 1996. Binary translation: static, dynamic, re-targetable?. In *1996 Proceedings of International Conference on Software Maintenance*. 340–349. <https://doi.org/10.1109/ICSM.1996.565037>
- [11] C. Cifuentes and M. Van Emmerik. 2000. UQBT: adaptable binary translation at low cost. *Computer* 33, 3 (2000), 60–66. <https://doi.org/10.1109/2.825697>
- [12] James C. Dehnert, Brian K. Grant, John P. Banning, Richard Johnson, Thomas Kistler, Alexander Klaiber, and Jim Mattson. 2003. The Transmeta Code Morphing™ Software: using speculation, recovery, and adaptive retranslation to address real-life challenges. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (San Francisco, California, USA) (CGO '03). IEEE Computer Society, USA, 15–24.
- [13] Artem Dinaburg and Andrew Ruef. 2025. Mcsema: Static translation of x86 instructions to llvm. In *ReCon 2014 Conference, Montreal, Canada*.
- [14] Alexis Engelke, Dominik Okwieka, and Martin Schulz. 2021. Efficient LLVM-based dynamic binary translation. In *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (Virtual, USA) (VEE 2021)*. Association for Computing Machinery, New York, NY, USA, 165–171. <https://doi.org/10.1145/3453933.3454022>
- [15] Alexis Engelke and Martin Schulz. 2020. Instrew: leveraging LLVM for high performance dynamic binary instrumentation. In *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (Lausanne, Switzerland) (VEE '20)*. Association

- for Computing Machinery, New York, NY, USA, 172–184. <https://doi.org/10.1145/3381052.3381319>
- [16] Alexis Engelke and Josef Weidendorfer. 2017. Using LLVM for Optimized Lightweight Binary Re-Writing at Runtime. In *2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 785–794. <https://doi.org/10.1109/IPDPSW.2017.103>
- [17] Haibing Guan, Erzhou Zhu, Hongxi Wang, Ruhui Ma, Yindong Yang, and Bin Wang. 2012. SINF: A dynamic-static combined framework for dynamic binary translation. *Journal of Systems Architecture* 58, 8 (2012), 305–317. <https://doi.org/10.1016/j.sysarc.2012.05.002>
- [18] Haoran Guo, Jianmin Pang, Yichi Zhang, Feng Yue, and Rongcai Zhao. 2010. HERO: A novel malware detection framework based on binary translation. In *2010 IEEE International Conference on Intelligent Computing and Intelligent Systems*, Vol. 1. 411–415. <https://doi.org/10.1109/ICICISYS.2010.5658586>
- [19] Laune C. Harris and Barton P. Miller. 2005. Practical analysis of stripped binary code. *SIGARCH Comput. Archit. News* 33, 5 (dec 2005), 63–68. <https://doi.org/10.1145/1127577.1127590>
- [20] Ding-Yong Hong, Chun-Chen Hsu, Pen-Chung Yew, Jan-Jan Wu, Wei-Chung Hsu, Pangfeng Liu, Chien-Min Wang, and Yeh-Ching Chung. 2012. HQEMU: a multi-threaded and retargetable dynamic binary translator on multicores. In *Proceedings of the Tenth International Symposium on Code Generation and Optimization* (San Jose, California) (CGO '12). Association for Computing Machinery, New York, NY, USA, 104–113. <https://doi.org/10.1145/2259016.2259030>
- [21] Ding-Yong Hong, Jan-Jan Wu, Yu-Ping Liu, Sheng-Yu Fu, and Wei-Chung Hsu. 2018. Processor-Tracing Guided Region Formation in Dynamic Binary Translation. *ACM Trans. Archit. Code Optim.* 15, 4, Article 52 (Nov. 2018), 25 pages. <https://doi.org/10.1145/3281664>
- [22] Ding-Yong Hong, Jan-Jan Wu, Pen-Chung Yew, Wei-Chung Hsu, Chun-Chen Hsu, Pangfeng Liu, Chien-Min Wang, and Yeh-Ching Chung. 2014. Efficient and Retargetable Dynamic Binary Translation on Multicores. *IEEE Transactions on Parallel and Distributed Systems* 25, 3 (2014), 622–632. <https://doi.org/10.1109/TPDS.2013.56>
- [23] Chun-Chen Hsu, Pangfeng Liu, Chien-Min Wang, Jan-Jan Wu, Ding-Yong Hong, Pen-Chung Yew, and Wei-Chung Hsu. 2011. LnQ: Building high performance dynamic binary translators with existing compiler backends. In *2011 International Conference on Parallel Processing*. IEEE, 226–234.
- [24] Huawei. 2025. About This Document-Introduction-ExaGear User Guide-Binary Translator (ExaGear)-Kunpeng Documentation-Kunpeng Community. https://www.hikunpeng.com/document/detail/en/kunpengdevps/ug-exagear/usermanual/kunpengexagear_06.html
- [25] Alexander C. Klaiber and Transmeta Corporation. 2000. The technology behind Crusoe processors. *Transmeta Technical Brief* (2000), 77–84.
- [26] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. 75–86. <https://doi.org/10.1109/CGO.2004.1281665>
- [27] Wei Li, Xiaohui Luo, Yiran Zhang, Qingkai Meng, and Fengyuan Ren. 2022. CrossDBT: An LLVM-Based User-Level Dynamic Binary Translation Emulator. In *Euro-Par 2022: Parallel Processing*, José Cano and Phil Trinder (Eds.). Springer International Publishing, Cham, 3–18.
- [28] Nicholas Nethercote and Julian Seward. 2007. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Diego, California, USA) (PLDI '07). Association for Computing Machinery, New York, NY, USA, 89–100. <https://doi.org/10.1145/1250734.1250746>
- [29] Trail of Bits. 2025. Trail of Bits | Open Source. <https://www.trailofbits.com/opensource/>
- [30] Nuno Paulino, João Bispo, João C. Ferreira, and João M. P. Cardoso. 2021. A Binary Translation Framework for Automated Hardware Generation. *IEEE Micro* 41, 4 (2021), 15–23. <https://doi.org/10.1109/MM.2021.3088670>
- [31] Mark Probst. 2002. Dynamic binary translation. In *UKUUG Linux Developer's Conference*, Vol. 2002.
- [32] LLVM Project. 2025. The LLVM Compiler Infrastructure Project. <https://llvm.org/>
- [33] LLVM Project. 2025. LLVM Language Reference Manual. <https://llvm.org/docs/LangRef.html>
- [34] LLVM Project. 2025. LLVM's Analysis and Transform Passes. <https://llvm.org/docs/Passes.html>
- [35] LLVM Project. 2025. ORC Design and Implementation. <https://llvm.org/docs/ORCv2.html>
- [36] Bor-Yeh Shen, Jiunn-Yeu Chen, Wei-Chung Hsu, and Wu Yang. 2012. LLBT: an LLVM-based static binary translator. In *Proceedings of the 2012 International Conference on Compilers, Architectures and Synthesis for Embedded Systems* (Tampere, Finland) (CASES '12). Association for Computing Machinery, New York, NY, USA, 51–60. <https://doi.org/10.1145/2380403.2380419>
- [37] Bor-Yeh Shen, Wei-Chung Hsu, and Wu Yang. 2014. A Retargetable Static Binary Translator for the ARM Architecture. *ACM Trans. Archit. Code Optim.* 11, 2, Article 18 (jun 2014), 25 pages. <https://doi.org/10.1145/2629335>
- [38] Bor-Yeh Shen, Jyun-Yan You, Wu Yang, and Wei-Chung Hsu. 2012. An LLVM-based hybrid binary translation system. In *7th IEEE International Symposium on Industrial Embedded Systems (SIES'12)*. 229–236. <https://doi.org/10.1109/SIES.2012.6356589>
- [39] Richard L Sites, Anton Chernoff, Matthew B Kirk, Maurice P Marks, and Scott G Robinson. 1993. Binary translation. *Commun. ACM* 36, 2 (1993), 69–81.
- [40] Maxwell Souza, Daniel Nicácio, and Guido Araújo. 2012. ISAMAP: Instruction Mapping Driven by Dynamic Binary Translation. In *Computer Architecture*, Ana Lucia Varbanescu, Anca Molnos, and Rob van Nieuwpoort (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 117–138.
- [41] Cheng Wang, Shiliang Hu, Ho-seop Kim, Sreekumar R. Nair, Mauricio Breternitz, Zhiwei Ying, and Youfeng Wu. 2007. StarDBT: An Efficient Multi-platform Dynamic Binary Translation System. In *Advances in Computer Systems Architecture*, Lynn Choi, Yunheung Paek, and Sangyeun Cho (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 4–15.
- [42] Zhaoxin Yang, Xuehai Chen, Liangpu Wang, Weiming Guo, Dongru Zhao, Chao Yang, and Fuxin Zhang. 2024. MFHBT: Hybrid Binary Translation System with Multi-stage Feedback Powered by LLVM. In *Advanced Parallel Processing Technologies*, Chao Li, Zhenhua Li, Li Shen, Fan Wu, and Xiaoli Gong (Eds.). Springer Nature Singapore, Singapore, 310–325.
- [43] Chuck Zhao, Youfeng Wu, J Gregory Steffan, and Cristiana Amza. 2008. Lengthening traces to improve opportunities for dynamic optimization. In *Proceedings of the Workshop on Interaction between Compilers and Computer Architectures*.